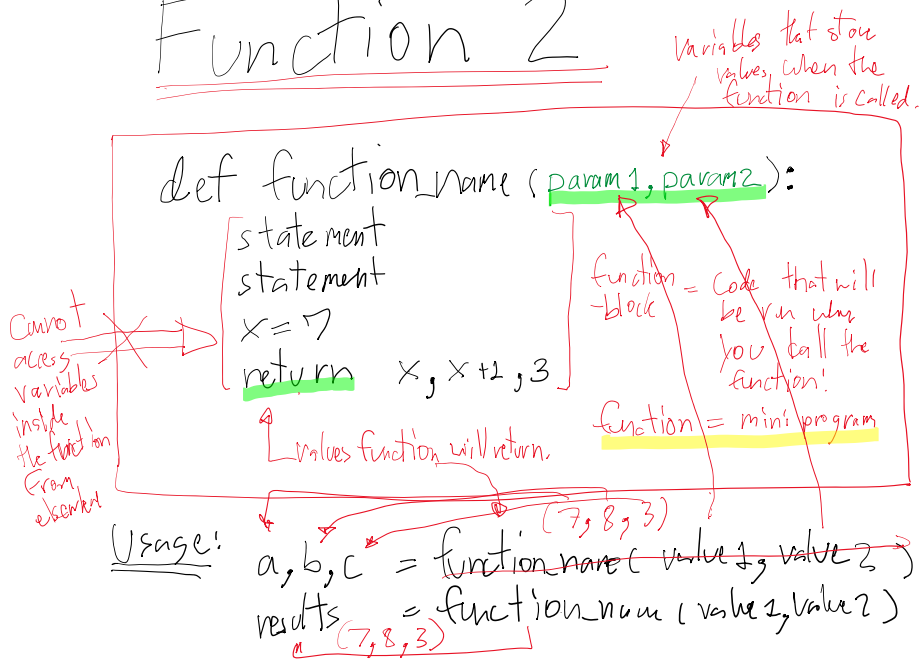


Function 2



How to think about function

1. Function will do one thing — mini program / sub question
2. Function can take in values. ← input into the function
↳ Not the same as input from the user! store in memory
3. Function can return values ← output of the function
↳ Not the same as print → output to the screen

Ex1 function to print my name

- 1) No param
- 2) No return

```
def print_name():
    print("Nur")
```

Ex2 function to print a name when you call the function

- 1) one param
- 2) No return

```
def print_name(name):
    print("Hello", name)
```

store a string

Usage: print_name("Alice")

Ex3 function to add two things and then print out

- 1) 2 params

```
def print_add(x, y):
    print(x+y)
```

store a number

and then print out
 1) 2 params
 2) No return

```
print (X+Y)
print_add(2,3)
```

Ex add two things
 and return the
 result

1) 2 params
 2) 1 return

```
def add(a,b)
    return a+b → 5
x = add(2,3) → 5
print(x) → 5
```

Exs BMI and
 obese or not
 (>30)

1) 2 params: w,h
 2) return 2 values
 BMI, "obese"

```
def BMI_cal(weight, height):
    bmi = weight/height**2
    if bmi > 30:
        return bmi, "obese"
    else:
        return bmi, "Not obese"
```

main() function

- no param
- no return
- get input from the user
- call BMI_cal(w,h)
- print out the result

```
def main():
    w = float(input("Enter weight: "))
    h = float(input("Enter height: "))
    bmi, obese = BMI_cal(w, h)
    print(f"bmi: {bmi} {obese}")

main()
```

local variable

Local Variables

x = 0 ← is different one!

```
def do_x():
    x = 5 ← local x only in do_x()
do_x()
print(x) → 0
```

Global Variables

Global Variables

Syntax: global variable_name *← do this inside a function!*

↳ make this variable global

↳ meaning that it becomes the same variable that is outside of the function

when you change this variable inside the function outside changes too.

```
x = 0  
def do_x():  
    global x  
    x = 5
```

same variable!

do_x()

print(x)

5

* Named Constant is still here!

↳ Sometimes we will call it, Global Constant

Library or Module

↳ Library/Module is a set of functions, typically created by others

↳ You need to import it to use its functions

Syntax: import library_name

Usage:

library_name.function_name(..)

Usage:

library_name.function_name(...)

Two libraries: 1) random
2) math

Random Library

- it has a bunch of functions to generate random numbers and do random sampling.

• Function: random.randint(lower, upper)

↳ it returns a random number between [lower, upper]

with equal probability. Inclusively!
include both.

random.randint(1, 5) $\xrightarrow[\text{one}]{\text{gen}}$ 1 2 3 4 5
prob $\frac{1}{5}$ $\frac{1}{5}$ $\frac{1}{5}$ $\frac{1}{5}$ $\frac{1}{5}$

EX import random
x = random.randint(1, 5)
print(x) $\xrightarrow{\text{random}}$ [?]

EX8 Data Analysis

- 1) gen_data(n) function $\rightarrow$ generate a list of random numbers between 1 and 10.
↳ take one value (n)
↳ return a list

→ take in a list

→ return a list

2) mean(numbers) function → cal mean of a given list of numbers

→ take in a list

→ return a number

3) main() function → will get n from the user and print out both data and mean

→ must call other functions.

import random

def gen_data(n):

my_list = []

for i in range(n):

x = random.randint(1, 10)

my_list.append(x)

return my_list

def mean(numbers):

eg. [4, 2, 3, 4]

return sum(numbers) / len(numbers)

def main():

n = int(input("Enter n: "))

data = gen_data(n)

mu = mean(data)

~~mean(numbers)~~

print(data)

print(mu)

main()

random.seed(int)

random.seed (int)

↳ set the random seed

↳ Same random seed will result in
Same sequence of random numbers.

math library

↳ provides math functions and
math constants

math functions

math.sin(x)

math.cos(x) [↑] radius

math.tan(x)

math.log(x)

math.exp(x) $\rightarrow e^x$

math.ceil(x) $\rightarrow \lceil x \rceil$

math.floor(x) $\rightarrow \lfloor x \rfloor$

math.abs(x) $\rightarrow |x|$

Constants

^{$\rightarrow \pi$}
math.pi = 3.14...

math.e = 2.71...
 $\rightarrow e$

Summary :

1) Functions

2) global variables

3) Library / Module

→ random
→ math